



Tecnológico Nacional de México

Instituto Tecnológico Superior de Felipe Carrillo Puerto

Ingeniería en Sistemas Computacionales



Asignatura:
Graficación

Tema:

1. Introducción a la graficación por computadora

EVIDENCIA DE APRENDIZAJE
AA 1.4 Guía de Evaluación práctica

Profesor:
MTI. Paloma Gongora Sabido

Alumno (s):
Chablé May Haniel Isaí
Sosa Alvarado Luis Angel
Tun Poot Jonathan

ISC - 3A

Felipe Carrillo Puerto a día 15 de Septiembre de 2026



2026
año de
**Margarita
Maza**



GUÍA TÉCNICA DE DESARROLLO GRÁFICO CON PROCESSING

En esta guía se muestra de manera práctica el uso de Processing para desarrollar gráficos mediante programación. Se explican los pasos básicos para instalar y configurar la herramienta, crear un lienzo, utilizar primitivas geométricas y modificar sus propiedades visuales.

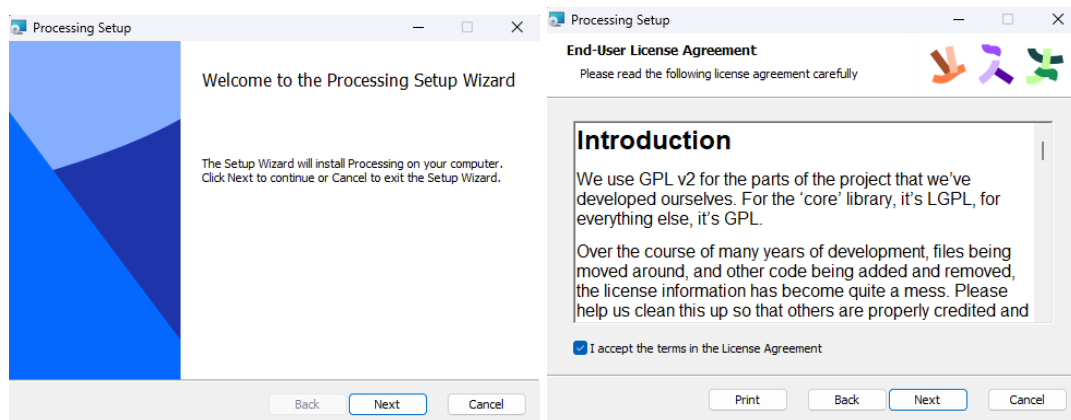
Como práctica se desarrollará un robot utilizando diferentes figuras geométricas. Al final se realiza una comparación entre ambas herramientas para identificar algunas diferencias en su sintaxis y forma de trabajo.

1. Instalación y configuración de Processing

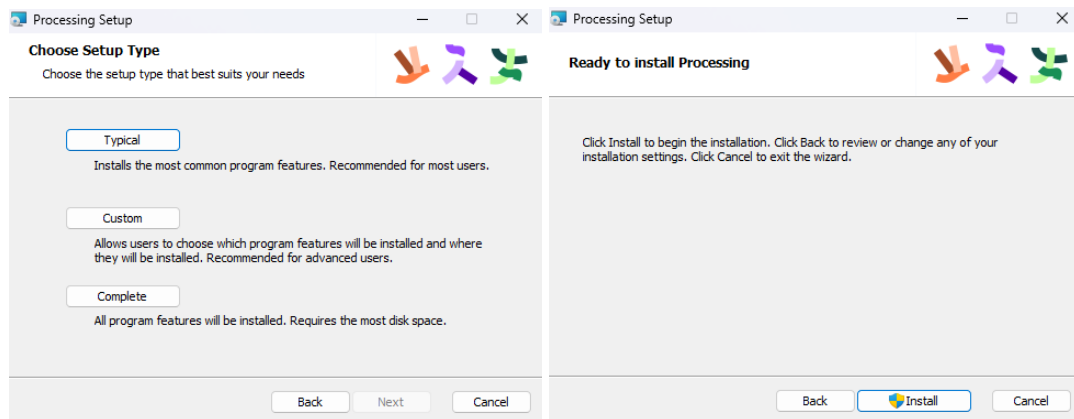
Para comenzar se debe descargar Processing desde su sitio oficial e instalar la versión correspondiente al sistema operativo:

https://processing.org/download?utm_source

Una vez instalado, se ejecuta el instalador y seleccionamos next. Posteriormente aceptamos los términos y condiciones, luego seleccionamos next:



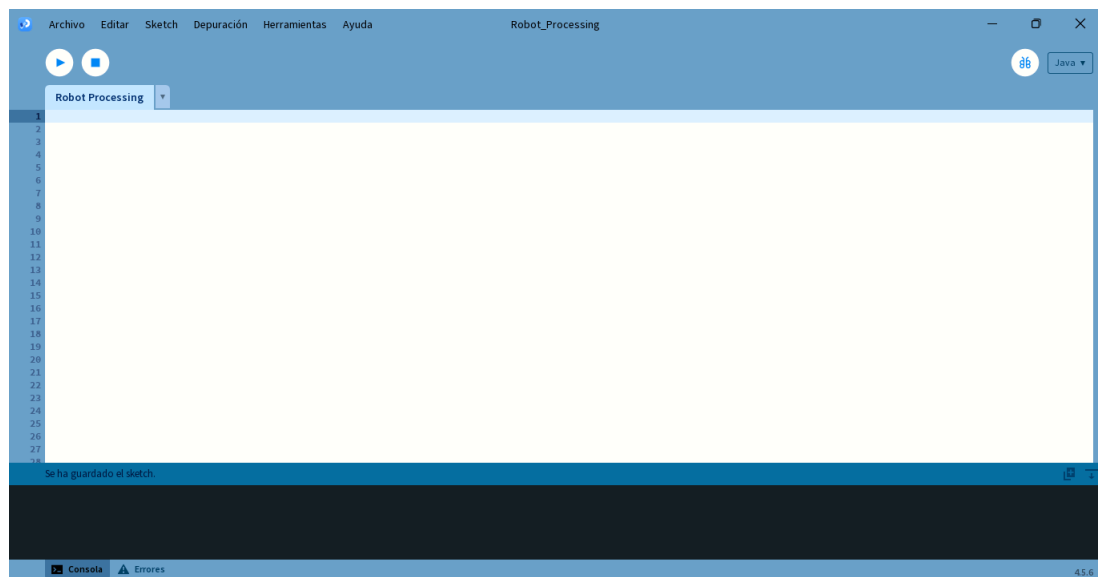
Por último seleccionamos la opción Typical y damos a next. Una vez hecho eso solo seleccionamos Install y una vez terminada la instalación seleccionamos la última opción que dice Finish:



Processing lo encontraremos en la siguiente dirección:

💻 Este equipo -> 💿 Disco local -> 📁 Archivos de programa -> 📁 Processing -> 🔄 Processing

Abrimos y creamos un nuevo Sketch, que es el espacio donde se escribirá y ejecutará el código. Para esta práctica el proyecto se guardó con el nombre: **Robot_Processing**



2. Creación y configuración del lienzo

Para crear el área donde se realizará el dibujo se utiliza la función `size()` dentro de `setup()`.

```
void setup() {  
    size(400, 400);  
}
```

En este caso se crea un lienzo de 400 píxeles de ancho por 400 píxeles de alto.

Para establecer el color de fondo se utiliza `background()`:
`background(25, 35, 45);`

Los valores corresponden al sistema de color RGB.

La estructura básica utilizada en el proyecto es:

```
void setup() {  
    size(400, 400);  
}  
  
void draw() {  
    background(25, 35, 45);  
}
```

`setup()` se ejecuta al iniciar el programa y `draw()` contiene las instrucciones de dibujo que se ejecutan mientras el programa está funcionando.

3. Primitivas geométricas

Processing permite crear gráficos utilizando diferentes primitivas geométricas. Para construir el robot se utilizaron principalmente:

Rectángulos

`rect(x, y, ancho, alto);`

Se utilizaron para formar la cabeza, el cuerpo, la pantalla, el panel y la sombra detrás de los ojos del robot.

Círculos

`circle(x, y, diámetro);`

Se utilizaron para los ojos, la punta de la antena y el botón del panel.

Líneas

`line(x1, y1, x2, y2);`

Se utilizaron para formar la antena y los brazos.

Triángulos

`triangle(x1, y1, x2, y2, x3, y3);`

Se utilizaron para formar las manos del robot.

Las coordenadas permiten establecer la posición de cada figura dentro del lienzo.

4. Colores, estilos y propiedades

Para modificar la apariencia de las figuras se utilizaron diferentes funciones.

Color de relleno: `fill(46, 204, 113);`

`fill()` establece el color del interior de una figura.

Color del contorno: `stroke(46, 204, 113);`

`stroke()` permite establecer el color del contorno.

Grosor del contorno: `strokeWeight(4);`

`strokeWeight()` modifica el grosor del contorno o de una línea.

Eliminar contorno: `noStroke();`

`noStroke()` permite dibujar las siguientes figuras sin contorno.

También se utilizó:

`rectMode(CENTER);`

`ellipseMode(CENTER);`

para utilizar el centro de las figuras como referencia de su posición.

Las propiedades de las figuras pueden modificarse directamente desde sus parámetros. Por ejemplo:

`circle(200, 265, 35);`

El primer y segundo valor representan la posición y el tercero el diámetro. Si se cambia `35` por `50`, el círculo aumenta de tamaño.

De la misma manera, modificar los valores de posición permite mover la figura y cambiar `strokeWeight()` permite modificar el grosor del contorno.

5. Elaboración del robot

Para realizar el robot se combinaron las primitivas y propiedades explicadas anteriormente.

El diseño se dividió en diferentes partes:

- Antena.
- Cabeza.
- Ojos.
- Brazos.
- Cuerpo.
- Panel frontal.
- Botón.
- Manos.

El código completo utilizado es:

```
void setup() {  
  size(400, 400);  
}
```

```
void draw() {  
  background(25, 35, 45);
```

```
  rectMode(CENTER);  
  ellipseMode(CENTER);
```

```
  // Antena  
  stroke(46, 204, 113);  
  strokeWeight(3);  
  line(200, 110, 200, 75);  
  line(200, 75, 215, 65);
```

```
  // Brazos  
  stroke(30, 132, 73);  
  strokeWeight(4);  
  line(130, 250, 95, 290);
```

```
line(270, 250, 305, 290);

noStroke();

// Punta de la antena
fill(46, 204, 113);
circle(215, 65, 12);

// Cabeza
fill(46, 204, 113);
rect(200, 140, 120, 80, 15);

// Pantalla de la cara
fill(15, 30, 20);
rect(200, 135, 90, 35, 8);

// Ojos
fill(235, 247, 240);
circle(180, 135, 16);
circle(220, 135, 16);

// Cuello
fill(27, 94, 32);
rect(200, 185, 30, 10);

// Cuerpo
fill(30, 132, 73);
rect(200, 270, 140, 160, 20);

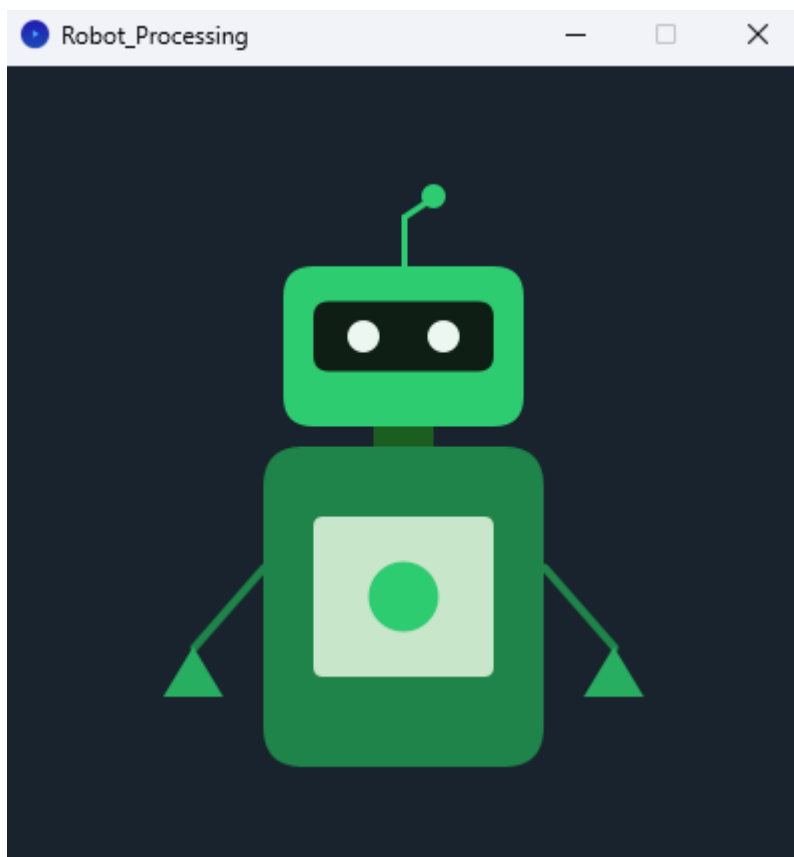
// Panel del cuerpo
fill(200, 230, 201);
rect(200, 265, 90, 80, 5);

// Botón
fill(46, 204, 113);
circle(200, 265, 35);
```



```
// Manos  
fill(39, 174, 96);  
triangle(95, 290, 80, 315, 110, 315);  
triangle(305, 290, 290, 315, 320, 315);  
}
```

El resultado obtenido es un robot construido completamente mediante código y primitivas geométricas.



6. Comparación con P5.js

Acción	Processing	P5.js
Crear lienzo	<code>size(400, 400);</code>	<code>createCanvas(400, 400);</code>
Función <code>setup</code>	<code>void setup()</code>	<code>function setup()</code>
Función <code>draw</code>	<code>void draw()</code>	<code>function draw()</code>
Relleno	<code>fill()</code>	<code>fill()</code>
Contorno	<code>stroke()</code>	<code>stroke()</code>
Grosor	<code>strokeWeight()</code>	<code>strokeWeight()</code>
Rectángulo	<code>rect()</code>	<code>rect()</code>
Círculo	<code>circle()</code>	<code>circle()</code>
Línea	<code>line()</code>	<code>line()</code>
Triángulo	<code>triangle()</code>	<code>triangle()</code>

Una de las principales diferencias observadas es la creación del lienzo. Processing utiliza `size()`, mientras que P5.js utiliza `createCanvas()`.

También cambia la sintaxis de las funciones principales, ya que Processing utiliza una sintaxis basada en Java, mientras que P5.js utiliza JavaScript.

A pesar de estas diferencias, las funciones relacionadas con el dibujo son muy similares, por lo que el robot pudo reproducirse utilizando instrucciones equivalentes.

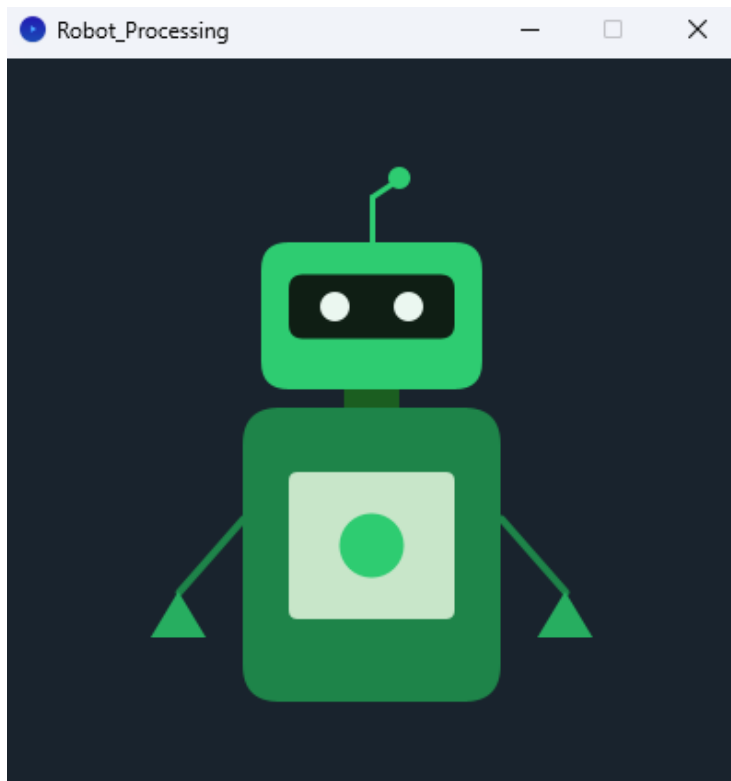
7. Evidencia de código fuente y Resultado obtenido

```
Robot Processing ▼
1 void setup() {
2   size(400, 400);
3 }
4
5 void draw() {
6   background(25, 35, 45);
7   rectMode(CENTER);
8   ellipseMode(CENTER);
9
10  // Antena
11  stroke(46, 204, 113);
12  strokeWeight(3);
13  line(200, 110, 200, 75);
14  line(200, 75, 215, 65);
15
16  // Brazos
17  stroke(30, 132, 73);
18  strokeWeight(4);
19  line(130, 250, 95, 290);
20  line(270, 250, 305, 290);
21  noStroke();
22
23  // Punta de la antena
24  fill(46, 204, 113);
25  circle(215, 65, 12);
26
27  // Cabeza
28  fill(46, 204, 113);
29  rect(200, 140, 120, 80, 15);
```

```

30
31 // Pantalla de la cara
32 fill(15, 30, 20);
33 rect(200, 135, 90, 35, 8);
34
35 // Ojos
36 fill(235, 247, 240);
37 circle(180, 135, 16);
38 circle(220, 135, 16);
39
40 // Cuello
41 fill(27, 94, 32);
42 rect(200, 185, 30, 10);
43
44 // Cuerpo
45 fill(30, 132, 73);
46 rect(200, 270, 140, 160, 20);
47
48 // Panel del cuerpo
49 fill(200, 230, 201);
50 rect(200, 265, 90, 80, 5);
51
52 // Botón
53 fill(46, 204, 113);
54 circle(200, 265, 35);
55
56 // Manos
57 fill(39, 174, 96);
58 triangle(95, 290, 80, 315, 110, 315);
59 triangle(305, 290, 290, 315, 320, 315);
60 }
61

```



GUÍA PARA INSTALAR VISUAL STUDIO CODE Y CREAR UN ROBOT CON HTML5 Y JAVASCRIPT

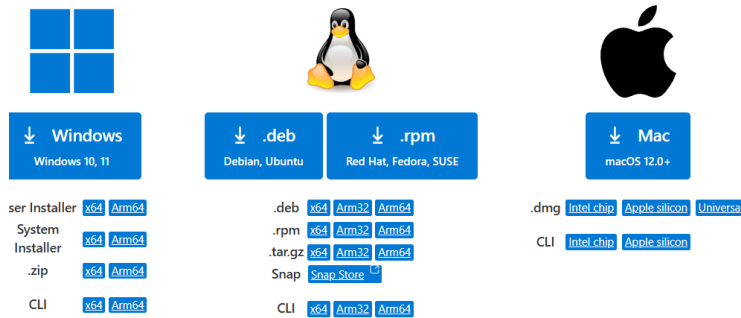
En esta guía se explica de manera sencilla cómo instalar Visual Studio Code, crear una carpeta para un proyecto de graficación y agregar los archivos necesarios para realizar un robot utilizando HTML5 y JavaScript. El proyecto se organiza en una carpeta llamada “graficacion”, donde se encuentran los archivos index.html y Scrip.js.

1. Instalación de Visual Studio Code

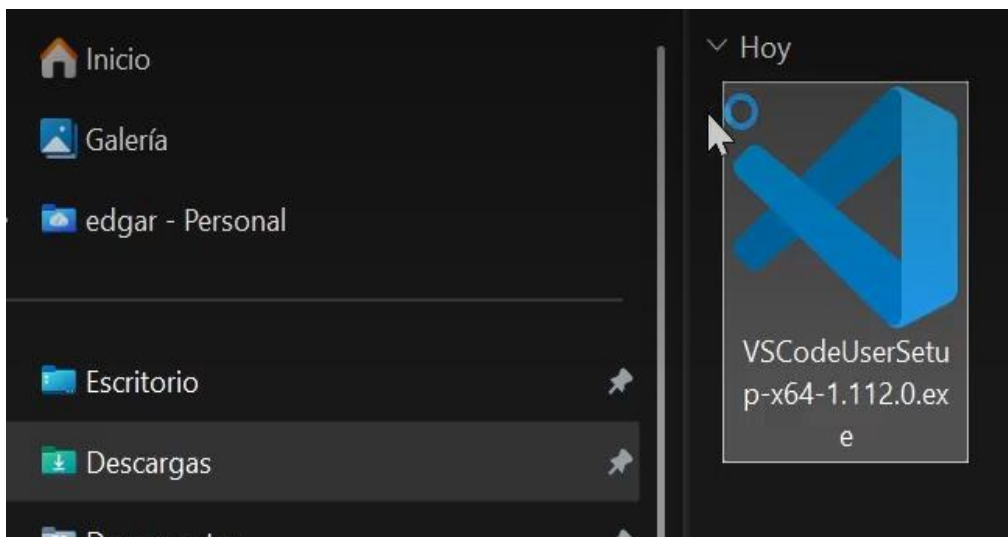
1. Ingresar al sitio oficial de Visual Studio Code y descargar el instalador correspondiente al sistema operativo.

Download Visual Studio Code

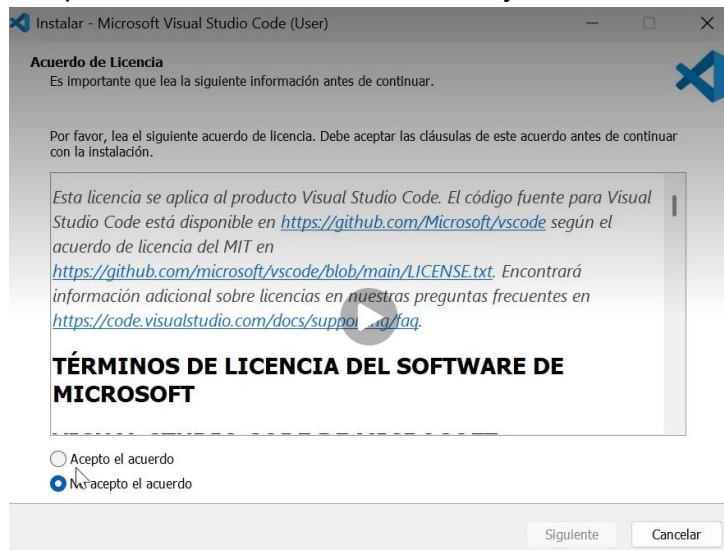
Free and built on open source. AI agents, Git, debugging, and extensions included.



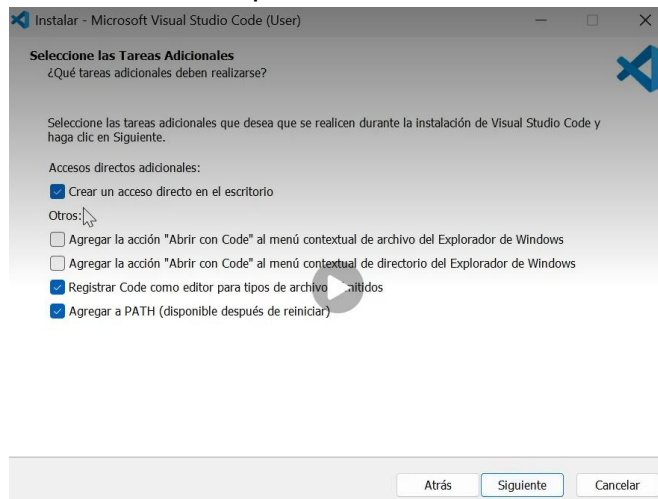
2. Ejecutar el archivo de instalación descargado.



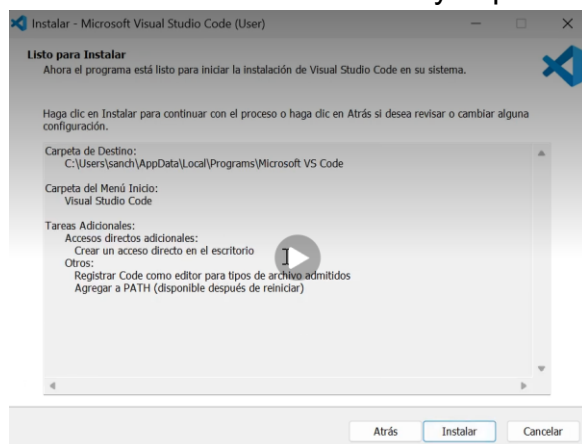
2. Aceptar los términos de la licencia y continuar con la instalación.



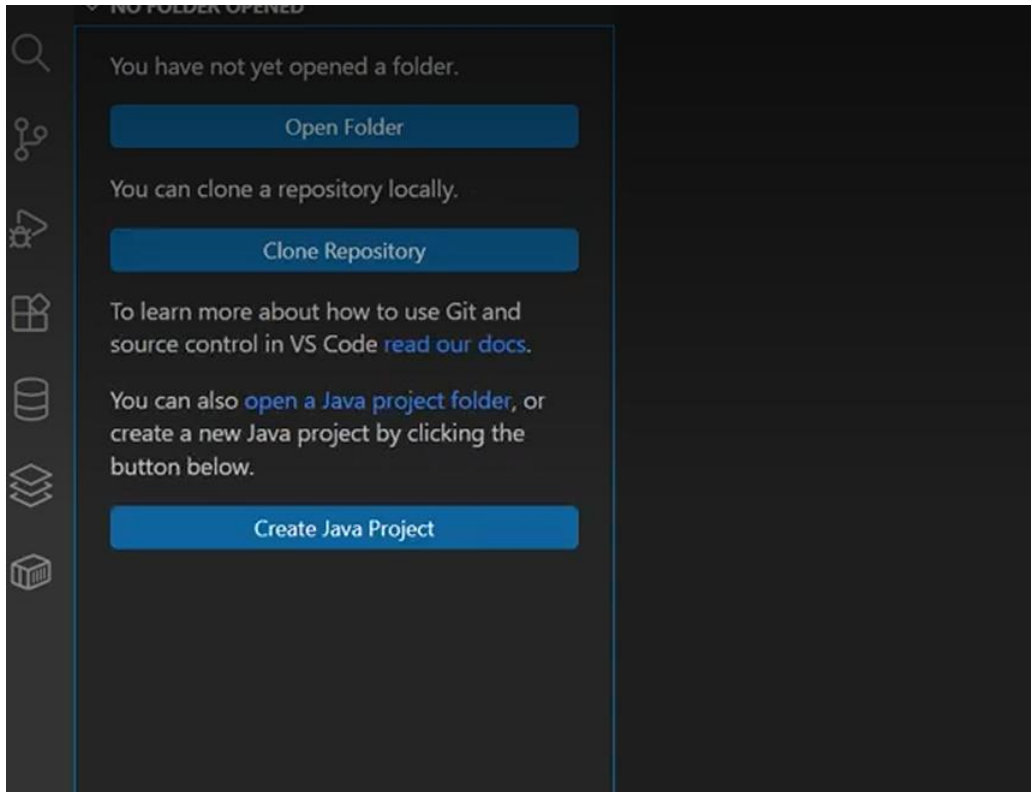
3. Seleccionar las opciones de instalación recomendadas.



4. Presionar el botón de instalar y esperar a que termine el proceso.

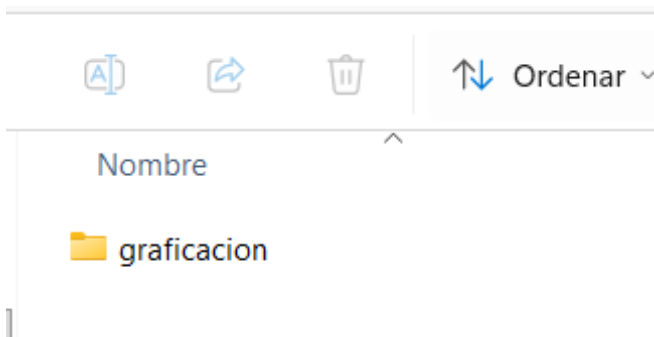


6. Abrir Visual Studio Code después de finalizar la instalación.



3. Crear la carpeta del proyecto

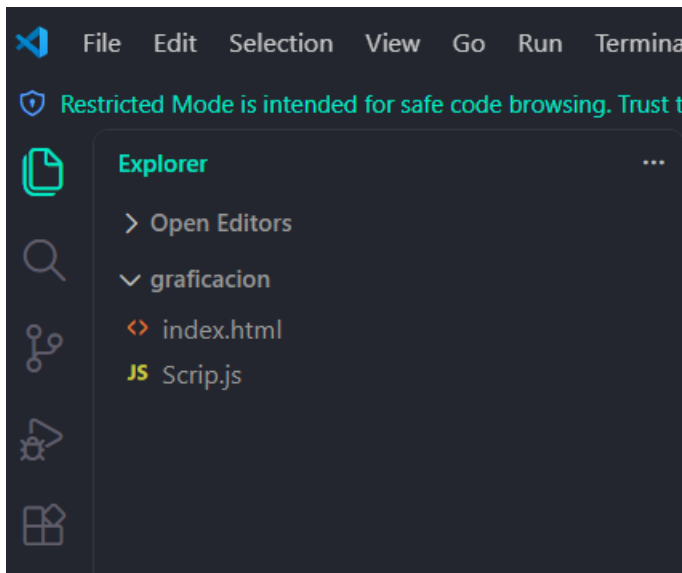
Para mantener los archivos organizados, se crea una carpeta llamada “graficacion”. Esta carpeta será utilizada para guardar todos los archivos relacionados con el robot.



4. Abrir la carpeta en Visual Studio Code

1. Abrir Visual Studio Code.
2. Seleccionar la opción Archivo > Abrir carpeta.
3. Buscar y seleccionar la carpeta “graficacion”.

4. Presionar “Seleccionar carpeta”. 5. La carpeta aparecerá en el panel Explorador de Visual Studio Code.



5. Crear el archivo index.html

Dentro de la carpeta “graficacion” se crea un archivo llamado index.html. Este será el archivo principal que se abrirá en el navegador.

Ejemplo de estructura inicial:

```
<> index.html > html
1  <!DOCTYPE html>
2  <html>
3  <head>
4    <title>Robot</title>
5  </head>
6  <body>
7
8    <title>Robot</title>
9
10   <canvas id="canvas" width="400" height="400"></canvas>
11
12   <Script src="Scrip.js"></Script>
13
14 </body>
15 </html>
```


6. Crear el archivo Scrip.js

Después se crea un segundo archivo llamado Scrip.js. En este archivo se coloca el código JavaScript encargado de dibujar el robot dentro del canvas.

Es importante que el nombre del archivo escrito en index.html coincida con el nombre del archivo JavaScript: Scrip.js.

```
JS Scrip.js
1 let canvas = document.getElementById("canvas");
2 let ctx = canvas.getContext("2d");
```

7. Ejecutar el proyecto

1. Guardar los archivos index.html y Scrip.js.
2. Abrir el archivo index.html en un navegador como Microsoft Edge, Google Chrome o Firefox.
3. El navegador mostrará el canvas y dentro de él aparecerá el robot creado con JavaScript.
4. Si el robot no aparece, revisar que los nombres de los archivos sean correctos y que index.html esté enlazando correctamente Scrip.js.

Nombre	Estado
 index	
 Scrip.js	

8. Explicación de código

moveTo() mueve el lápiz sin dibujar.

lineTo() dibuja una línea.

closePath() cierra la figura.

Figura	Comando
Cuadrado	fillRect()

Figura	Comando
Rectángulo	fillRect()
Círculo	arc()
Triángulo	moveTo() + lineTo()
Línea	moveTo() + lineTo()

9. codigo de Robot

```
S Scrip.js
1  let canvas = document.getElementById("canvas");
2  let ctx = canvas.getContext("2d");
3  // fondo
4  ctx.fillStyle = "□rgb(25, 35, 45)";
5  ctx.fillRect(0, 0, 400, 400);
6  // antena
7  ctx.strokeStyle = "■rgb(46, 204, 113)";
8  ctx.lineWidth = 3;
9
10 ctx.beginPath();
11 ctx.moveTo(200, 110);
12 ctx.lineTo(200, 75);
13 ctx.lineTo(215, 65);
14 ctx.stroke();
15
16 // punta antena
17 ctx.fillStyle = "■rgb(46, 204, 113)";
18
19 ctx.beginPath();
20 ctx.arc(215, 65, 6, 0, Math.PI * 2);
21 ctx.fill();
22 // cabeza
23
24 ctx.fillStyle = "■rgb(46, 204, 113)";
25
26 ctx.beginPath();
27 ctx.roundRect(140, 100, 120, 80, 15);
28 ctx.fill();
29
30 //pantalla
31 ctx.fillStyle = "□rgb(15, 30, 20)";
32
33 ctx.beginPath();
34 ctx.roundRect(155, 117.5, 90, 35, 8);
35 ctx.fill();
```

```
37 // ojos
38 ctx.fillStyle = "■rgb(235, 247, 240)";
39
40 ctx.beginPath();
41 ctx.arc(180, 135, 8, 0, Math.PI * 2);
42 ctx.fill();
43
44 ctx.beginPath();
45 ctx.arc(220, 135, 8, 0, Math.PI * 2);
46 ctx.fill();
47 //cuello
48
49 ctx.fillStyle = "■rgb(15, 113, 15)";
50
51 ctx.fillRect(185, 180, 30, 10);
52
53
54 // CUERPO
55
56
57 ctx.fillStyle = "■rgb(30, 132, 73)";
58
59 ctx.beginPath();
60 ctx.roundRect(130, 190, 140, 160, 20);
61 ctx.fill();
62
63
```

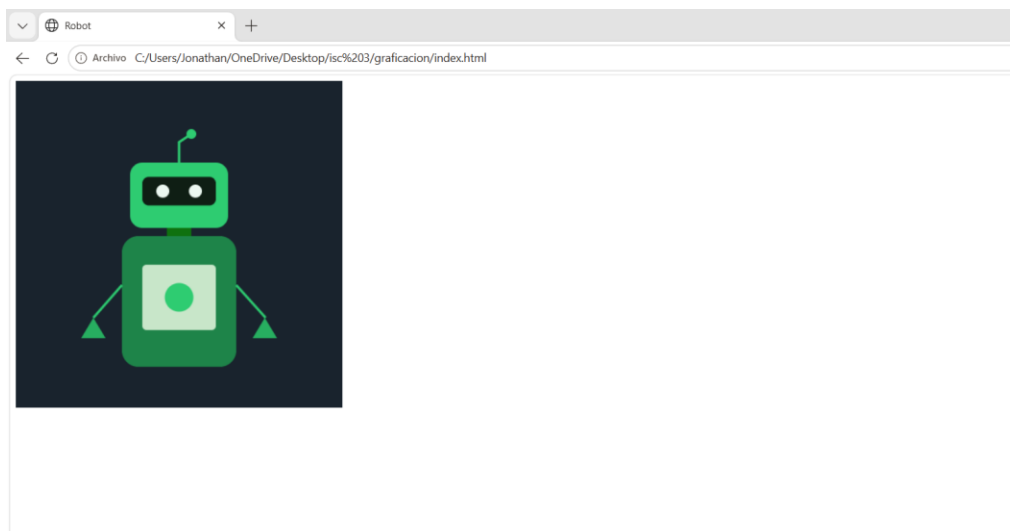
```
4 // panel del cuerpo
5
6 ctx.fillStyle = "■rgb(200, 230, 201)";
7
8 ctx.beginPath();
9 ctx.roundRect(155, 225, 90, 80, 5);
9 ctx.fill();
1
2 //centro del cuerpo
3 ctx.fillStyle = "■rgb(46, 204, 113)";
4
5 ctx.beginPath();
6 ctx.arc(200, 265, 17.5, 0, Math.PI * 2);
7 ctx.fill();
8
9 ctx.fillStyle = "■rgb(39, 174, 96)";
9
1 // Mano izquierda
2 ctx.beginPath();
3 ctx.moveTo(95, 290);
4 ctx.lineTo(80, 315);
5 ctx.lineTo(110, 315);
6 ctx.closePath();
7 ctx.fill();
8
9 // Mano derecha
9 ctx.beginPath();
1 ctx.moveTo(305, 290);
2 ctx.lineTo(290, 315);
3 ctx.lineTo(320, 315);
4 ctx.closePath();
5 ctx.fill()
```

```

81 // Mano izquierda
82 ctx.beginPath();
83 ctx.moveTo(95, 290);
84 ctx.lineTo(80, 315);
85 ctx.lineTo(110, 315);
86 ctx.closePath();
87 ctx.fill();
88
89 // Mano derecha
90 ctx.beginPath();
91 ctx.moveTo(305, 290);
92 ctx.lineTo(290, 315);
93 ctx.lineTo(320, 315);
94 ctx.closePath();
95 ctx.fill();
96
97 // brazo
98 ctx.fillStyle = "■rgb(39, 174, 96)";
99 ctx.lineWidth = 3;
100 ctx.beginPath();
101 ctx.moveTo(270, 250);
102 ctx.lineTo(306, 290);
103 ctx.stroke();
104
105 ctx.beginPath();
106 ctx.moveTo(130, 250);
107 ctx.lineTo(95, 290);
108 ctx.stroke();

```

10. Resultado de Robot

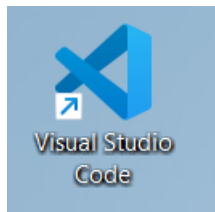


Guía de CSS

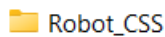
Para realizar la práctica se utilizó Visual Studio Code, un editor de código que permite crear y modificar archivos HTML y CSS. En este entorno se organizaron los archivos del proyecto y se escribieron las instrucciones necesarias para generar una figura gráfica mediante estilos CSS.

1. Pasos para preparar el entorno

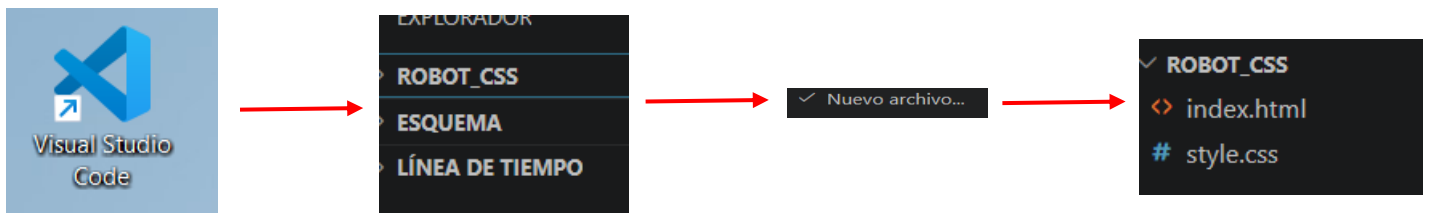
- Abrir VisualStudiocode.



- Crear una carpeta nueva en tu computadora en este caso sería Robot_CSS



- Ingresamos otra vez a visualStudio code y dentro de la carpeta de Robot_CSS crearemos dos archivos porque CSS no puede crear elementos desde cero. Necesita que el HTML cree las "cajas" (<div>) y luego CSS las transforma en figuras.



2. ¿Cómo funciona la "Compilación" en CSS?

- CSS es un lenguaje interpretado directamente por el navegador. No existe un paso intermedio de compilación. El navegador lee el archivo .css, lo entiende y lo aplica al HTML en tiempo real.
- Para que el navegador sepa que debe leer el archivo CSS, es obligatorio colocar esta etiqueta dentro del <head> del archivo index.html:

```
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Robot en CSS</title>
7   <link rel="stylesheet" href="style.css">
8 </head>
9 <body>
```

3. Cómo colocar el código para ejecutarlo

- Escribir la estructura en HTML en el archivo index.html, se crean los "moldes" (cajas <div>) de cada parte del robot:

```
<div class="lienzo">

  <div class="antena-palo"></div>
  <div class="antena-codo"></div>
  <div class="antena-bola"></div>

  <div class="cuello"></div>

  <div class="brazo izquierdo"></div>
  <div class="brazo derecho"></div>

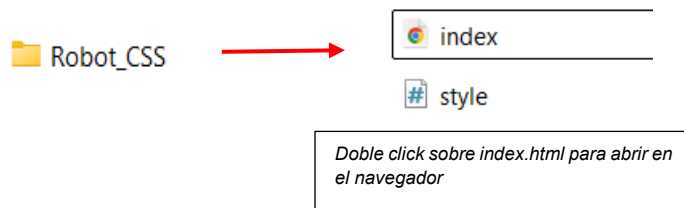
  <div class="cabeza">
    <div class="visor">
      <div class="ojo izquierdo"></div>
      <div class="ojo derecho"></div>
    </div>
  </div>

  <div class="cuerpo">
    <div class="panel-pecho"></div>
    <div class="circulo-pecho"></div>
  </div>
```

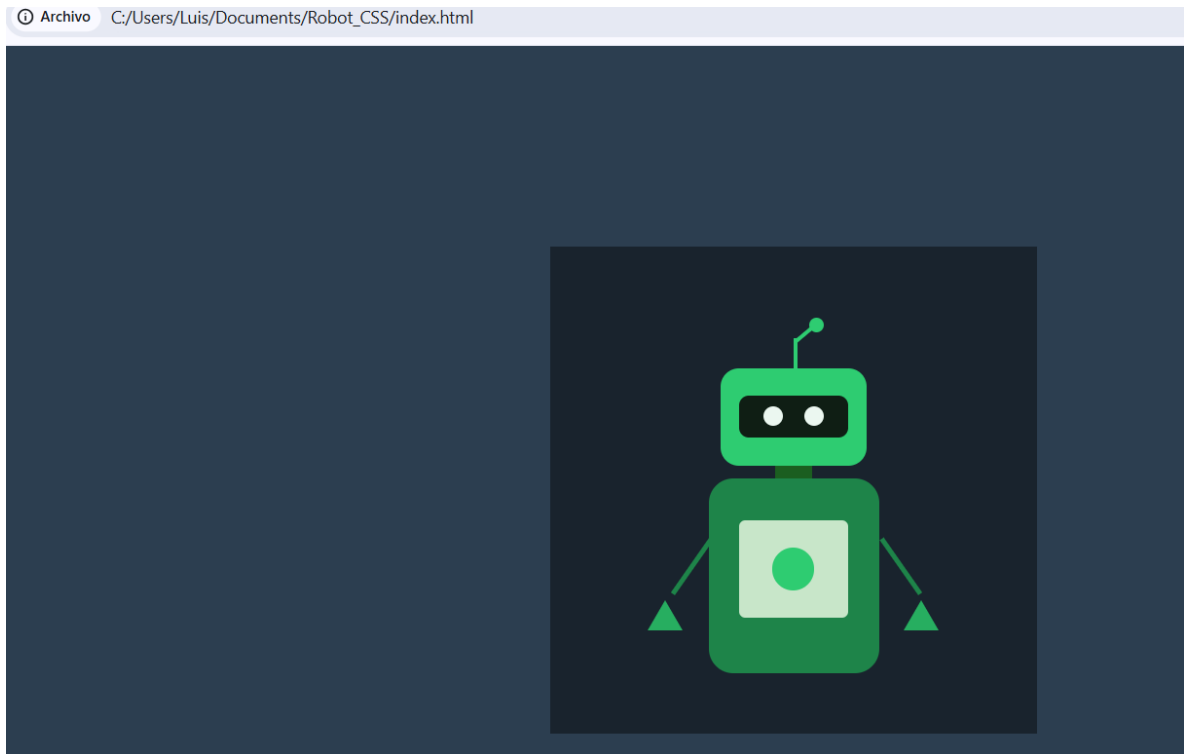
- Escribir los estilos en css en el archivo style.css, se le da forma, color y posición a cada molde

```
2  body {
3      margin: 0;
4      display: flex;
5      justify-content: center;
6      align-items: center;
7      height: 100vh;
8      background-color: #2c3e50;
9  }
10
11  .lienzo {
12      position: relative;
13      width: 400px;
14      height: 400px;
15      background-color: rgb(25, 35, 45);
16      overflow: hidden;
17  }
18
19
20  .antena-palo {
21      position: absolute;
22      width: 3px;
23      height: 40px;
24      background-color: rgb(46, 204, 113);
25      left: 200px;
26      top: 75px;
27  }
28
29  .antena-codo {
30      position: absolute;
31      width: 3px;
32      height: 18px;
33      background-color: rgb(46, 204, 113);
34      left: 200px;
35      top: 60px;
36      transform: rotate(50deg);
37      transform-origin: bottom center;
```

- Guardar y Ejecutar ambos archivos (Ctrl + S en VS Code).
- Abrir el archivo index.html en el navegador



- El navegador mostrara al robot en pantalla




```
<div class="cuerpo">
  <div class="panel-pecho"></div>
  <div class="circulo-pecho"></div>
</div>

<div class="mano izquierda"></div>
<div class="mano derecha"></div>
</div>
</body>
</html>
```

Style.css

```
# style.css > .antena-palo
1
2  body {
3      margin: 0;
4      display: flex;
5      justify-content: center;
6      align-items: center;
7      height: 100vh;
8      background-color: #2c3e50;
9  }
10
11  .lienzo {
12      position: relative;
13      width: 400px;
14      height: 400px;
15      background-color: rgb(25, 35, 45);
16      overflow: hidden;
17  }
18
19
20  .antena-palo {
21      position: absolute;
22      width: 3px;
23      height: 40px;
24      background-color: rgb(46, 204, 113);
25      left: 200px;
26      top: 75px;
27  }
28
29  .antena-codo {
30      position: absolute;
31      width: 3px;
32      height: 18px;
33      background-color: rgb(46, 204, 113);
34      left: 200px;
35      top: 60px;
36      transform: rotate(50deg);
```

```
37     transform-origin: bottom center;
38 }
39
40 .antena-bola {
41     position: absolute;
42     width: 12px;
43     height: 12px;
44     background-color: ■ rgb(46, 204, 113);
45     border-radius: 50%;
46     left: 213px;
47     top: 58px;
48 }
49
50
51 .cuello {
52     position: absolute;
53     width: 30px;
54     height: 15px;
55     background-color: ■ rgb(27, 94, 32);
56     left: 185px;
57     top: 175px;
58 }
59
60
61 .cabeza {
62     position: absolute;
63     width: 120px;
64     height: 80px;
65     background-color: ■ rgb(46, 204, 113);
66     border-radius: 15px;
67     left: 140px;
68     top: 100px;
69 }
```

```
70
71 .visor {
72     position: absolute;
73     width: 90px;
74     height: 35px;
75     background-color: #rgb(15, 30, 20);
76     border-radius: 8px;
77     left: 15px;
78     top: 22px;
79 }
80
81
82 .ojo {
83     position: absolute;
84     width: 16px;
85     height: 16px;
86     background-color: #rgb(235, 247, 240);
87     border-radius: 50%;
88     top: 9px;
89 }
90
91 .ojo.izquierdo { left: 20px; }
92 .ojo.derecho { right: 20px; }
93
94
95 .cuerpo {
96     position: absolute;
97     width: 140px;
98     height: 160px;
99     background-color: #rgb(30, 132, 73);
100     border-radius: 20px;
101     left: 130px;
102     top: 190px;
103 }
```



```
105
106 .panel-pecho {
107     position: absolute;
108     width: 90px;
109     height: 80px;
110     background-color: ■ rgb(200, 230, 201);
111     border-radius: 5px;
112     left: 25px;
113     top: 35px;
114 }
115
116
117 .circulo-pecho {
118     position: absolute;
119     width: 35px;
120     height: 35px;
121     background-color: ■ rgb(46, 204, 113);
122     border-radius: 50%;
123     left: 52px;
124     top: 57px;
125 }
126
127
128 .brazo {
129     position: absolute;
130     width: 4px;
131     height: 55px;
132     background-color: ■ rgb(30, 132, 73);
133     top: 240px;
134     transform-origin: top center;
135 }
136
137 .brazo.izquierdo {
138     left: 130px;
139     transform: rotate(35deg);
140 }
```

```
141
142 .brazo.derecho {
143     left: 270px;
144     transform: rotate(-35deg);
145 }
146
147
148 .mano {
149     position: absolute;
150     width: 0;
151     height: 0;
152     border-left: 15px solid transparent;
153     border-right: 15px solid transparent;
154     border-bottom: 25px solid #263238;
155     top: 290px;
156 }
157
158 .mano.izquierda { left: 80px; }
159 .mano.derecha { left: 290px; }
```

Se adjuntan dos archivos: index.html (estructura) y style.css (estilos). El uso de HTML se limita exclusivamente a la creación de los "moldes" o cajas contenedoras, ya que CSS es un lenguaje de estilos y no puede generar elementos por sí solo. La totalidad del diseño gráfico (lienzo, primitivas geométricas, colores, posiciones y transformaciones) se implementó con CSS, tal como corresponde a mi rol como Integrante 3.